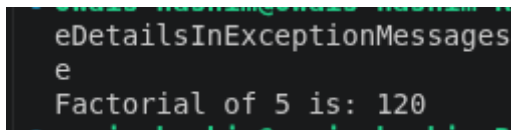


Name : Muhammad Hussain
Reg No : FA24-BCS-121
Instructor : Sir Ehaz Mustafa
Lab #18/19:

#Recursive Function :

```
public class recursive {  
    public int factorial(int x){  
        if(x==1){  
            return 1;}  
        else{  
            return x * factorial(x - 1);}}  
    public static void main(String[] args) {  
        recursive obj = new recursive();  
        int result = obj.factorial(5);  
        System.out.println("Factorial of 5 is: " + result);}}
```

Output :



```
eDetailsInExceptionMessages  
e  
Factorial of 5 is: 120
```

#BST :

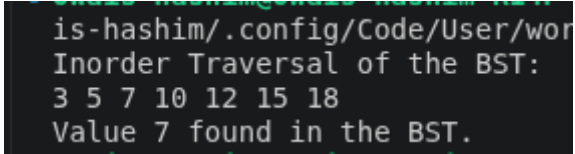
```
public class BST {  
    class Node {  
        int data;  
        Node left;  
        Node right;  
        public Node(int data) {  
            this.data = data;  
            left = null;  
            right = null;}}  
    private Node root;  
    public BST() {  
        root = null;}  
    public void insert(int data) {  
        Node newNode = new Node(data);  
        if (root == null) {  
            root = newNode;  
            return;}  
        Node current = root;  
        Node parent;  
        while (true) {  
            parent = current;  
            if (data < current.data) {  
                current = current.left;  
                if (current == null) {  
                    parent.left = newNode;  
                    return;}  
                }  
            }  
            if (data > current.data) {  
                current = current.right;  
                if (current == null) {  
                    parent.right = newNode;  
                    return;}  
                }  
            }  
        }  
    }  
}
```

```

return;}
} else {
current = current.right;
if (current == null) {
parent.right = newNode;
return;}}}}
// Search for a value
public boolean find(int id) {
Node current = root;
while (current != null) {
if (current.data == id) {
return true;
} else if (id < current.data) {
current = current.left;
} else {
current = current.right;}}
return false;}
// Inorder Traversal
public void display(Node root) {
if (root != null) {
display(root.left);
System.out.print(root.data + " ");
display(root.right);}}
// Getter for root
public Node getRoot() {
return root;}
public static void main(String[] args) {
BST b = new BST();
b.insert(10);
b.insert(5);
b.insert(15);
b.insert(3);
b.insert(7);
b.insert(12);
b.insert(18);
System.out.println("Inorder Traversal of the BST:");
b.display(b.getRoot());
int searchValue = 7;
if (b.find(searchValue)) {
System.out.println("\nValue " + searchValue + " found in the BST.");
} else {
System.out.println("\nValue " + searchValue + " not found in the BST.");}}}

```

OUTPUT :



```

is-hashim/.config/Code/User/wor
Inorder Traversal of the BST:
3 5 7 10 12 15 18
Value 7 found in the BST.

```

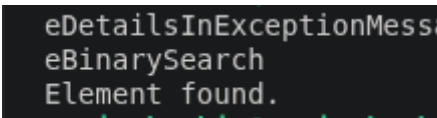
Binary Search Algorithm :

```
class TreeNode {
    int data;
    TreeNode left, right;
    TreeNode(int data) {
        this.data = data;
        left = right = null;
    }
}

public class RecursiveBinarySearch {
    TreeNode root;
    public boolean binarySearch(TreeNode root, int key) {
        if (root == null)
            return false;
        if (root.data == key)
            return true;
        if (key < root.data)
            return binarySearch(root.left, key);
        return binarySearch(root.right, key);
    }
    public static void main(String[] args) {
        RecursiveBinarySearch tree = new RecursiveBinarySearch();
        tree.root = new TreeNode(50);
        tree.root.left = new TreeNode(30);
        tree.root.right = new TreeNode(70);
        tree.root.left.left = new TreeNode(20);
        tree.root.left.right = new TreeNode(40);
        tree.root.right.left = new TreeNode(60);
        tree.root.right.right = new TreeNode(80);

        if (tree.binarySearch(tree.root, 60))
            System.out.println("Element found.");
        else
            System.out.println("Element not found.");
    }
}
```

OUTPUT :



```
eDetailsInExceptionMess
eBinarySearch
Element found.
```